# Minimum Difference Between BST Nodes (easy)

## Problem Statement

Given a Binary Search Tree (BST), you are required to find the smallest difference between the values of any two different nodes.

In a BST, the nodes are arranged in a manner where the value of nodes on the left is less than or equal to the root, and the value of nodes on the right is greater than the root.

### Example

*Example 1:*

- Input:

```
    4
   / \
  2   6
 / \
1   3
```

- Expected Output: 1
- Justification: The pairs (1,2), (2,3), or (3,4) have the smallest difference of 1.

*Example 2:*

- Input:

```
    10
   /  \
  5   15
 / \    \
2   7    18
```

- Expected Output: 2
- Justification: The pair (5,7) has the smallest difference of 2.

*Example 3:*

- Input:

```
  40
   \
    70
   /  \
  50  90
```

- Expected Output: 10
- Justification: The pair (40,50) has the smallest difference of 10.

## Try it yourself

Try solving this question here:

Java

```java
import java.util.ArrayList;
import java.util.List;

// TreeNode class to represent the Binary Search Tree nodes.
// class TreeNode {
//     int val;
//     TreeNode left;
//     TreeNode right;
//     TreeNode(int x) { val = x; }
// }

public class Solution {
    // Using a List to hold the values in order.
    private List<Integer> nodes = new ArrayList<>();

    public int minDiffInBST(TreeNode root) {
        int minDiff = Integer.MAX_VALUE;
        // ToDo: Write Your Code Here.
        return minDiff;
    }
}
```

Test Case | Run Code Result | Stdout

[4,2,6,1,3]

Use Example Testcases | Run | Submit